
ΚΑΤΑΝΕΜΗΜΕΝΑ ΣΥΣΤΗΜΑΤΑ

2^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ:

Υλοποίηση στατικής και δυναμικής χρονοδρομολόγησης

ΟΜΑΔΑ 5380-5226

Κωνσταντίνος Ζώης, AM: 5226, e-mail: cs05226@uoi.gr

Ιωάννης Φίλλης, AM: 5380, e-mail: cs05380@uoi.gr

ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

ΜΑΪΟΣ 2026

1 ΕΙΣΑΓΩΓΗ

Σε αυτή την άσκηση πρέπει να παραλληλοποιήσουμε ένα απλό πρόγραμμα, υλοποιώντας μέσω του MPI στατική και δυναμική χρονοδρομολόγηση. Θα τρέξουμε τον κώδικα σε διάφορους κόμβους των υπολογιστών του εργαστηρίου, οι οποίοι αποτελούν ένα κατανεμημένο σύστημα.

Σκοπός είναι να συγκρίνουμε την συμπεριφορά των δύο αυτών μεθόδων και στο τέλος θα παρουσιάσουμε τα συμπεράσματά μας και θα τα αναλύσουμε με κάποιες γραφικές παραστάσεις.

2 ΥΛΟΠΟΙΗΣΗ

Παρακάτω περιγράφουμε τα βήματα που ακολουθήσαμε για να υλοποιήσουμε σε κώδικα τις δύο μεθόδους.

2.1 ΣΤΑΤΙΚΗ ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΣΗ

Για την στατική χρονοδρομολόγηση, στο αρχείο **static-sched.c** προφανώς το πρώτο που έχουμε είναι η συνάρτηση `work()`, όπου αυτή είναι η “δουλειά” που κάνουν. Έπειτα, στην `main()` είναι όλη η λογική. Ξεκινάμε με τις βασικές ρουτίνες **MPI_INIT()**, **MPI_Comm_rank()**, **MPI_Comm_size()** και η πρώτη διεργασία, με rank 0, διαβάζει από την είσοδο το `n` και το κάνει εκπομπή και στις υπόλοιπες. Έπειτα, έχουμε την **MPI_Barrier()**, για να περιμένουμε όλες τις διεργασίες στο σημείο που θα ξεκινήσει η χρονομέτρηση.

Μετά γίνονται οι υπολογισμοί, για το κομμάτι που θα αναλάβει η κάθε διεργασία. Στην περίπτωση όπου περισσεύουν δουλειές, τότε ανατίθενται στις πρώτες διεργασίες, από μία δουλειά στην καθεμία. Το πως υπολογίζεται η τιμή της μεταβλητής `start` εξαρτάται από ποια περίπτωση είναι. Η πρώτη περίπτωση είναι η διεργασία να ανήκει σε αυτές που (πιθανόν) παίρνουν μία έξτρα δουλειά. Όλες αυτές παίρνουν $(\text{chunk}+1)$ δουλειές. Αν υπάρχουν και άλλες διεργασίες πριν από αυτήν (δηλαδή με μικρότερο rank) τότε και αυτές πήραν $(\text{chunk}+1)$ δουλειές. Επίσης γενικά για μία διεργασία με αναγνωριστικό rank, υπάρχουν rank διεργασίες πριν από αυτήν. Άρα, το πόσες επαναλήψεις θα προσπεράσουμε προκύπτει από τον τύπο $\text{rank} * (\text{chunk}+1)$ και άρα $(\text{rank} * \text{chunk} + \text{rank})$ σε αυτή την περίπτωση. Η δεύτερη περίπτωση διεργασίας που μπορεί να έχουμε είναι το rank της να είναι μεγαλύτερο ή ίσο από `remaining`, οπότε εκεί οι προηγούμενες διεργασίες ανήκουν σε 2 κατηγορίες. Υπάρχουν `remaining` διεργασίες που πήραν `chunk+1` δουλειές και `rank - remaining` διεργασίες που πήραν `chunk` δουλειές. Οπότε, για να βρούμε το `start`, αθροίζουμε τις δύο περιπτώσεις, δηλαδή θα πρέπει να ισούται με $\text{remaining} * (\text{chunk}+1) + (\text{rank} - \text{remaining}) * \text{chunk}$, όπου με απλοποίηση προκύπτει $\text{rank} * \text{chunk} + \text{remaining}$.

Η καθεμία από τις διεργασίες, δεσμεύει τον δικό της χώρο προκειμένου να αποθηκεύει τοπικά τα αποτελέσματά της. Αμέσως μετά, η καθεμία από αυτές κάνει τους υπολογισμούς με τις δικές της, τοπικές μεταβλητές **start, end**.

Ακολούθως, ο κώδικας χωρίζεται στον κώδικα που εκτελεί η διεργασία 0, η οποία συλλέγει τα αποτελέσματα και στον κώδικα που εκτελούν οι υπόλοιποι όπου στέλνουν τα αποτελέσματα στην διεργασία 0.

Ακριβώς από κάτω, καλούμε την **MPI_Barrier()**, και μετράμε ξανά τον χρόνο για να υπολογίσουμε το τέλος των υπολογισμών. Έπειτα, η διεργασία 0 εκτυπώνει τον χρόνο και όλες κάνουν free τον χώρο που δέσμευσαν και καλούν την **MPI_Finalize()** για να τερματίσουν.

2.2 ΔΥΝΑΜΙΚΗ ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΣΗ

Σε αυτήν την περίπτωση, η διεργασία 0 έχει τον ρόλο του master. Έχουμε δημιουργήσει 3 tags: το TAG_TASK που χρησιμοποιείται για να σηματοδοτήσει ανάθεση δουλειάς, το TAG_RESULT για να σηματοδοτήσει επιστροφή αποτελέσματος και το TAG_DIE που λειτουργεί ως εντολή τερματισμού. Ξανά αρχικά έχουμε τις 3 βασικές ρουτίνες. Η διεργασία 0 διαβάζει από την είσοδο το n. Ξανά, για την χρονομέτρηση χρησιμοποιούμε την εντολή **MPI_Barrier()**.

Έπειτα, ο κώδικας χωρίζεται στον κώδικα που θα εκτελέσει ο master και στον κώδικα που θα εκτελέσει ο worker. Δεσμεύουμε χώρο για την δομή **result** που αποθηκεύουμε το αποτέλεσμα και επίσης την μεταβλητή **next_task**, για να μπορούμε να δούμε αν έχουμε διαθέσιμες άλλες δουλειές για να ανατεθούν και την **active_workers** για να κρατάμε το πλήθος των διεργασιών που κάνουν δουλειά. Αρχικά λοιπόν, ο master στέλνει από μία δουλειά σε κάθε διεργασία. Έπειτα, μπαίνει σε ένα loop όπου περιμένει αποτέλεσμα από οποιαδήποτε διεργασία και αν υπάρχουν και άλλες δουλειές που απέμειναν και δεν ανατέθηκαν στέλνει σε αυτήν μία δουλειά, αλλιώς την ενημερώνει κατάλληλα ότι πρέπει να τερματίσει. Για να μπορέσουν να συγχρονιστούν καλά, ο master με τους workers, έχουμε προσθέσει και δύο buffers, έναν στην πλευρά του master και ένα στους workers. Οι workers δεν στέλνουν μόνο το αποτέλεσμα αλλά και την ερώτηση. Έτσι, ο master λαμβάνει έναν πίνακα 2 θέσεων όπου στην πρώτη είναι ο αριθμός της επανάληψης που εκτέλεσε ο worker (πχ 4) και στην δεύτερη θέση το αποτέλεσμα (πχ 16). Έτσι, ο master μπορεί να διαχειριστεί σωστά τις απαντήσεις και να τις βάλει στην σωστή σειρά.

Από κάτω προσθέσαμε ένα κομμάτι κώδικα όπου σε περίπτωση όπου έχουμε λιγότερες δουλειές από ότι διεργασίες τότε θα δεσμευθούν αλλά δεν θα κάνουν δουλειά, οπότε σε αυτήν την περίπτωση τους στέλνει μήνυμα να τερματίσουν. Στο τέλος εκτυπώνει το αποτέλεσμα και αποδεσμεύει την μνήμη του result.

Στην περίπτωση των εργατών, μπαίνουν σε ένα while loop που τρέχει διαρκώς και λαμβάνουν δουλειές, κάνουν την δουλειά τους και στέλνουν το αποτέλεσμα. Φεύγουν από το loop μόνο όταν λάβουν το TAG για να τερματίσουν.

Στο τέλος πάλι με την **MPI_Barrier()** συγχρονίζονται όλες οι διεργασίες για να είναι σωστός ο χρόνος τερματισμού και ο master υπολογίζει τον χρόνο και τον εκτυπώνει. Καλείται και η **MPI_Finalize()** για να τερματίσουν οι διεργασίες.

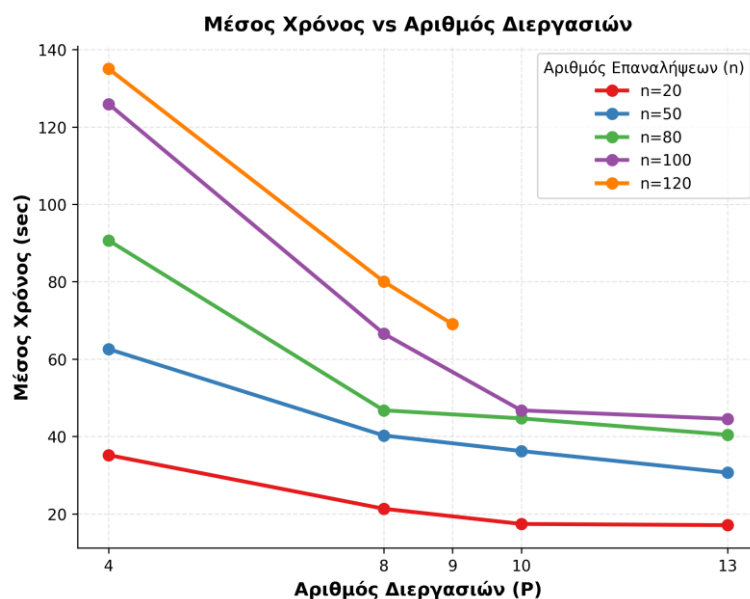
3 ΑΠΟΤΕΛΕΣΜΑΤΑ ΚΑΙ ΣΥΜΠΕΡΑΣΜΑΤΑ

Σε αυτό το σημείο παρουσιάζουμε τα πειράματα που τρέξαμε χρονομετρώντας την επίδοση των υλοποιήσεων για διαφορετικές τιμές των αριθμών επαναλήψεων και διεργασιών. Ξεκινάμε από 20 επαναλήψεις και τέλος φτάνουμε στις 120. Δοκιμάσαμε αρχικά 4 διεργασίες και τελικά 13, ώστε να χρησιμοποιούνται όλοι οι κόμβοι των μηχανημάτων που έχουμε ορίσει στο hostfile. Στην περίπτωση των 120 επαναλήψεων, εκτελέσαμε τα πειράματα σε μεταγενέστερο χρόνο όπου ο τρίτος υπολογιστής του hostfile ήταν εκτός λειτουργίας και για αυτό ο αριθμός διεργασιών φτάνει μόνο μέχρι τις 9. Για να είναι στατιστικά αποδεκτός ο χρόνος, τρέξαμε το κάθε πείραμα από 5 φορές και υπολογίσαμε τον μέσο όρο των χρόνων εκτέλεσης.

3.1 ΣΤΑΤΙΚΗ ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΣΗ

Τα αποτελέσματα του μέσου χρόνου εκτέλεσης της στατικής χρονοδρομολόγησης για κάθε πείραμα φαίνονται στον παρακάτω πίνακα.

Αριθμός Επαναλήψεων	Αριθμός Διεργασιών	Μέσος Χρόνος Εκτέλεσης (sec)
20	4	35.169
20	8	21.305
20	10	17.370
20	13	17.052
50	4	62.561
50	8	40.238
50	10	36.214
50	13	30.645
80	4	90.649
80	8	46.743
80	10	44.654
80	13	40.420
100	4	125.919
100	8	66.565
100	10	46.731
100	13	44.531
120	4	135.011
120	8	80.025
120	9	69.011



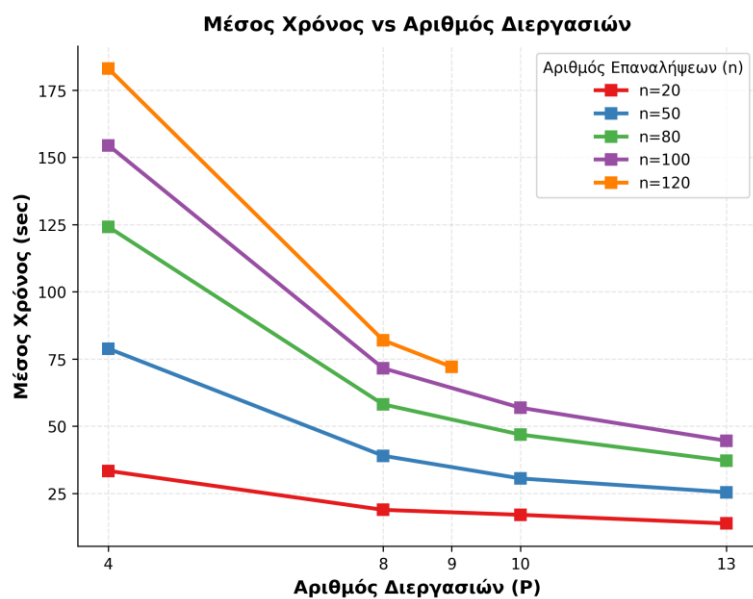
Από το παραπάνω γράφημα βλέπουμε φυσικά ότι όσο αυξάνεται ο αριθμός των επαναλήψεων, αυξάνεται και ο χρόνος εκτέλεσης. Όταν αυξάνονται οι διεργασίες ο χρόνος μειώνεται, αφού οι επαναλήψεις διαμοιράζονται σε περισσότερους κόμβους και επιτυγχάνεται περισσότερος παραλληλισμός. Από 4 σε 8 κόμβους συγκεκριμένα, σε κάθε περίπτωση ο χρόνος μειώνεται περίπου στο μισό, ενώ από 10 σε 13 είναι ελάχιστη η διαφορά ίσως επειδή με 13 χρησιμοποιούμε όλους τους κόμβους που έχουμε διαθέσιμους, αλλά δεν διαιρούνται ακριβώς με τον αριθμό επαναλήψεων και κάποιες διεργασίες παίρνουν παραπάνω επαναλήψεις.

3.2 ΔΥΝΑΜΙΚΗ ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΣΗ

Στον παρακάτω πίνακα φαίνονται τα αποτελέσματα του μέσου χρόνου εκτέλεσης της δυναμικής χρονοδρομολόγησης για κάθε πείραμα που εκτελέσαμε.

Αριθμός Επαναλήψεων	Αριθμός Διεργασιών	Μέσος Χρόνος Εκτέλεσης (sec)
20	4	33.396
20	8	18.909
20	10	17.060
20	13	13.845
50	4	78.852
50	8	39.033
50	10	30.589
50	13	25.451
80	4	124.165
80	8	58.146
80	10	46.903
80	13	37.146
100	4	154.486
100	8	71.587
100	10	56.895
100	13	44.617
120	4	183.018
120	8	82.022
120	9	72.084

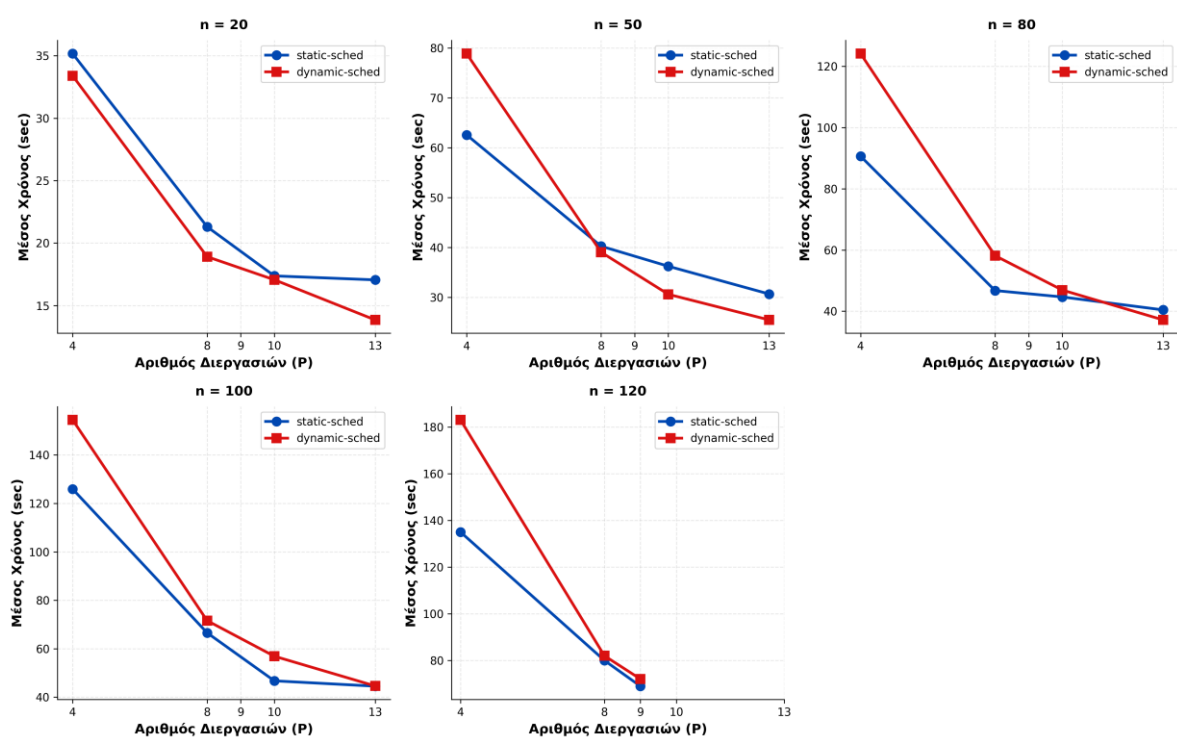
Στο παρακάτω γράφημα παρατηρούμε παρόμοια αποτελέσματα με την στατική χρονοδρομολόγηση ως προς τον μέσο χρόνο συγκριτικά με τον αριθμό διεργασιών για κάθε διαφορετικό n . Ξανά από 4 σε 8 διεργασίες ο χρόνος πέφτει σχεδόν στο μισό και εδώ υπάρχει μεγαλύτερη διαφορά στη μείωση του χρόνου από 10 σε 13 διεργασίες, ιδιαίτερα για τις περιπτώσεις $n=80$ και $n=100$.



3.3 ΣΥΓΚΡΙΣΗ ΜΕΘΟΔΩΝ

Σε αυτό το σημείο θα συγκρίνουμε τα αποτελέσματα των πειραμάτων που πήραμε για τις δύο μεθόδους και θα προσπαθήσουμε να βγάλουμε κάποια συμπεράσματα για την συμπεριφορά τους.

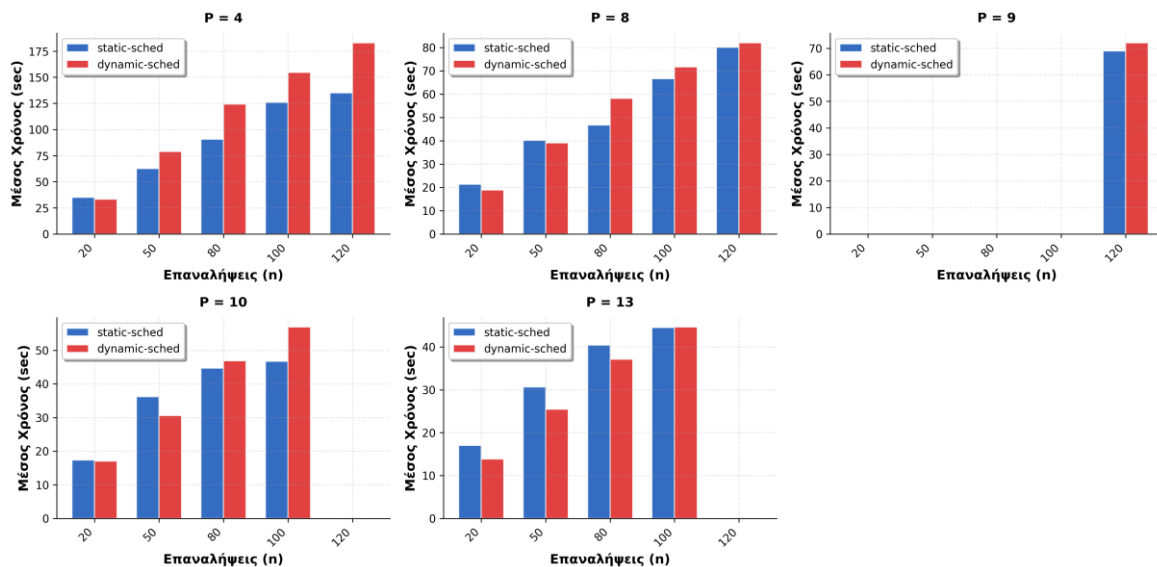
**Σύγκριση Στατικής vs Δυναμικής Χρονοδρομολόγησης
ανά αριθμό επαναλήψεων n**



Στο παραπάνω γράφημα παρουσιάζουμε συγκριτικά την στατική και την δυναμική χρονοδρομολόγηση σε κάθε διαφορετικό αριθμό επαναλήψεων n ως προς την επίδοση τους σε μεταβλητό αριθμό διεργασιών. Βλέπουμε ότι η δυναμική δεν είναι πάντα καλύτερη και σε πολλές περιπτώσεις η στατική χρονοδρομολόγηση έχει καλύτερη απόδοση. Αρχικά, για πιο μικρά n ($n=20, 50$) η δυναμική έχει γενικά καλύτερο χρόνο, εκτός από την περίπτωση όπου $P=4$ για 50 επαναλήψεις. Αυτό συμβαίνει επειδή οι 50 επαναλήψεις σε σχέση με τις 20 αυξάνουν παραπάνω από διπλάσιο χρόνο το sleep άρα και τον χρόνο εκτέλεσης και στην δυναμική χρονοδρομολόγηση η πρώτη διεργασία έχει τον ρόλο του συντονιστή και έτσι μόνο 3 διεργασίες θα κάνουν δουλειά σε αυτή την περίπτωση. Σε αυτό μπορεί να οφείλεται που η δυναμική είναι αρκετά πιο αργή στις 4 διεργασίες σε κάθε άλλη περίπτωση των n . Στα υπόλοιπα μεγαλύτερα n όμως ($n=80, 100, 120$) η στατική χρονοδρομολόγηση είναι πάντα ταχύτερη εκτός από την περίπτωση για $n=80, P=13$. Για τις μεγαλύτερες τιμές του n και P λοιπόν, η στατική είναι αρκετά καλύτερη στις περιπτώσεις για ($n=80, P=8$), ($n=100, P=8$) και ($n=100, P=10$) ενώ στις περιπτώσεις ($n=80, P=10$), ($n=120, P=8$) και ($n=120, P=9$) είναι ελαφρώς καλύτερη.

Στα μεγαλύτερα n λοιπόν, η στατική χρονοδρομολόγηση φαίνεται να έχει καλύτερη απόδοση. Αυτό μπορεί να οφείλεται στο ότι η δυναμική προσθέτει πολύ μεγαλύτερο overhead στην επικοινωνία. Στη στατική μέθοδο ένας κόμβος ενημερώνει αρχικά τους υπόλοιπους για τον αριθμό των επαναλήψεων, οι κόμβοι αποφασίζουν για την κατανομή των επαναλήψεων και στη συνέχεια ο καθένας εκτελεί αυτές που του αναλογούν και στέλνει το αποτέλεσμα πίσω. Έχουμε δηλαδή $2 \cdot (P-1)$ ανταλλαγές μηνυμάτων (ο κόμβος 0 στέλνει το n σε όλους τους υπόλοιπους, οι υπόλοιποι στέλνουν το αποτέλεσμα πίσω στον 0). Στην δυναμική όμως, ο κόμβος συντονιστής στέλνει τις επαναλήψεις μία-μία στους υπόλοιπους κόμβους (εργάτες), κάθε φορά που ένας εργάτης τελειώσει την εκτέλεση μίας επανάληψης στέλνει το αποτέλεσμα πίσω και ο συντονιστής θα του δώσει μία νέα επανάληψη. Αυτός ο δυναμικός διαμοιρασμός των επαναλήψεων επιφέρει συνολικά περίπου $2 \cdot n$ ανταλλαγές μηνυμάτων (ο συντονιστής στέλνει συνολικά n επαναλήψεις στους εργάτες και οι εργάτες στέλνουν συνολικά n αποτελέσματα πίσω στον συντονιστή). Επομένως για μεγάλα n , στην δυναμική χρονοδρομολόγηση γίνονται πολύ περισσότερες ανταλλαγές μηνυμάτων απ' ότι στην στατική.

Σύγκριση Στατικής vs Δυναμικής Χρονοδρομολόγησης
για Διαφορετικό Αριθμό Διεργασιών P



Και από το παραπάνω γράφημα βλέπουμε την απόδοση των μεθόδων για κάθε διαφορετικό αριθμό διεργασιών. Φαίνεται καθαρά ότι στις μεγάλες τιμές του n , η στατική χρονοδρομολόγηση υπερβαίνει έναντι της δυναμικής. Μόνο στην περίπτωση με 13 διεργασίες η δυναμική υπερβαίνει την στατική ακόμα και για 80 επαναλήψεις και στις 100 έχουν περίπου ίδιο χρόνο.

Συμπερασματικά, όπως είδαμε δεν μπορούμε να ξεχωρίσουμε μία από τις δύο μεθόδους ως καλύτερη. Η μία μέθοδος υπερβαίνει της άλλης σε συγκεκριμένες περιπτώσεις ανάλογα με τις παραμέτρους των πειραμάτων.